

Read-First, Role-Gated Context Federation Beats Point-to-Point Message-Passing for Multi-Agent LLM Systems: A Fairness-Controlled Evaluation

Adrian Issac Mamani Quispe
Universidad Nacional de San Agustín de Arequipa
Arequipa, Peru
admamaniq@unsa.edu.pe

Ryan Fabian Valdivia Segovia
Universidad Nacional de San Agustín de Arequipa
Arequipa, Peru
rvaldiviase@unsa.edu.pe

June 2026

Abstract

Multi-agent systems built on large language models (LLMs) overwhelmingly coordinate by *message-passing*: each agent sends task and context directly to named peers, as in Google’s Agent2Agent (A2A) protocol and the classical agent communication languages it descends from. We argue that an alternative substrate—*read-first, role-gated context federation* (the Graph Context Protocol, GCP)—is structurally superior on three axes at equal task success: **coupling**, **information leakage**, and **cost**. In GCP, an agent does not push context to peers; it *reads* context from independently owned knowledge nodes through a policy gate that authorizes the read by role and records a provenance entry for every decision. We make the comparison *fair* by holding everything constant except the interop layer: both arms run the identical agent brain, the identical model, and the identical task; only the per-peer tool factory and node hosting differ (an in-process GCP fetch handler versus real localhost A2A servers). On three scenarios—a marketplace (coupling), a software organization (leakage), and a cross-owner supply chain—we find: (i) the number of pairwise integration edges grows as $\mathcal{O}(n^2)$ for A2A versus $\mathcal{O}(n)$ for GCP (2450 vs. 50 at $n=50$), with constant marginal integration effort for GCP; (ii) provenance completeness is 1.0 for GCP and 0.0 for A2A across all scenarios and seeds, because read decisions are audited by construction whereas point-to-point messages leave no access trail; and (iii) GCP never costs more, and on the cross-owner task costs fewer tokens with far lower variance (707.6 ± 3.5 vs. 901.4 ± 229.2). Task success is equal across arms. We position GCP against five decades of coupling theory and access-control research and against recent LLM-agent security work, and we re-

lease the harness, scenarios, and seeds. The evaluation is preliminary—one free-tier model, five seeds—and we are explicit about what that does and does not license.

Keywords—*context federation; multi-agent systems; large language models; access control; least privilege; provenance; software coupling; agent communication protocols*

I Introduction

The dominant coordination pattern for LLM-based multi-agent systems is direct message-passing. Frameworks such as AutoGen [1], ChatDev [2], and MetaGPT [3] compose applications from agents that send and receive messages, and the emerging Agent2Agent (A2A) protocol [4] standardizes exactly this: a client agent advertises and delegates tasks to named remote agents over JSON-RPC. This paradigm is a direct descendant of the classical agent communication languages KQML [5] and FIPA-ACL [6], which already assumed pairwise connections and shared, pre-negotiated protocols between every interacting pair.

Message-passing has a structural cost that the software-engineering literature has understood for fifty years. Connecting n components pairwise requires on the order of n^2 links; introducing a shared intermediary that each component connects to once reduces this to n links. This is the “integration spaghetti” that the Enterprise Integration Patterns [7] catalog addresses with message brokers and hubs, and it is the same phenomenon that coupling theory [8–10] formalizes: a system’s modifiability is governed by the number and strength of its inter-component dependencies.

We study an alternative substrate for multi-agent coordination, the Graph Context Protocol (GCP). GCP is

read-first: an agent obtains what it needs by reading context from independently owned knowledge nodes, rather than receiving pushed messages. It is *role-gated*: every read passes through a policy gate that authorizes the request by the caller’s role and required capabilities, defaulting to deny [11, 12]. And it is *audited*: every read decision—grant or deny—produces a structured provenance record naming the principal, target node, decision, and matched policy.

Thesis. At equal task success, read-first role-gated context federation beats point-to-point message-passing on coupling, leakage, and cost.

Contributions.

1. **Architecture.** We describe GCP: read-first, role-gated context federation with first-class per-read provenance (Section III), and contrast it with push-based message-passing.
2. **A fairness-controlled benchmark method.** The single most common flaw in architecture comparisons is that the arms differ in more than the architecture. Our harness builds both arms from one scenario definition and one injected model, swapping *only* the interop layer (Section V).
3. **An empirical head-to-head** over three scenarios with a real LLM and five seeds, reporting counts (connections, messages, tokens), not just wall-clock (Section VI).
4. **Metric definitions** for structural coupling, canary exact-match leakage (a lower bound), and provenance completeness (Section IV).

We are deliberately conservative about the empirical claims: the behavioral results come from a single free-tier model over five seeds and are best read as a *pilot*. Section VII states the threats to validity in full, including a case where the architectural leak in GCP’s favor is *latent rather than observed* because the model never exercised the leaky path.

II Background and Related Work

A Message-passing as the multi-agent baseline

Agent communication languages established the message-passing paradigm: KQML [5] wrapped knowledge payloads in typed performatives exchanged over pairwise connections, and FIPA-ACL [6] standardized a 12-field message in which sender and receiver must share content language and ontology before any exchange succeeds. Modern LLM frameworks reinvented this in natural language. AutoGen [1] programs applications as conversations between

agents; ChatDev [2] arranges role-pairs in a fixed “chat chain”; and surveys of the field [13] taxonomize communication structure into peer-to-peer, centralized, layered, and shared-pool variants. The Model Context Protocol [14] standardizes agent-to-tool context access (an early read-first form), while A2A [4] standardizes agent-to-agent task delegation—the baseline we benchmark against. Recent work has begun to quantify message-passing’s overhead: Zhang et al. [15] report 28–73% excess tokens in inter-agent communication graphs and recover most of it by pruning the message topology. Most directly relevant, Mao and Mirhoseini [16] replace both centralized orchestration and peer-to-peer routing with a shared verified context and report state-of-the-art results at roughly half the cost—evidence that a read-from-shared-context substrate can dominate message-passing. We add to that line role-gated authorization, audited per-read provenance, and a fairness-controlled head-to-head.

B Coupling and the n^2 problem

That fewer inter-component links yield more maintainable systems is among the oldest results in software engineering. Parnas [8] showed that information-hiding decomposition—exposing only a minimal interface—produces dramatically lower coupling than flowchart-style partitioning. Stevens et al. [9] introduced coupling and cohesion as measurable, ordinal properties of inter-module dependency. Briand et al. [10] gave a graph-theoretic, measurement-theoretic framework in which coupling is a monotone function of dependency-edge count, so an $\mathcal{O}(n)$ -edge topology is provably less coupled than an $\mathcal{O}(n^2)$ one. Hohpe and Woolf [7] translated this to distributed integration: a full point-to-point mesh of n systems needs $n(n-1)/2$ channels, which a hub-and-spoke broker reduces to n . Microservice research [17] carries the same edge-count argument into cloud-native systems, and in the LLM setting Shen et al. [18] show empirically that dense, fully-connected agent topologies amplify error propagation while sparser topologies suppress it. We apply this directly: A2A creates pairwise agent $\leftrightarrow$ agent edges; GCP creates agent $\rightarrow$ store edges.

C Least privilege, RBAC, and gated context

GCP’s read gate operationalizes the principle of least privilege, articulated by Saltzer and Schroeder [11] and equated there with military need-to-know. Role-based access control [12] decouples permissions from individuals by assigning them to roles, and attribute-based access control [19] generalizes the policy to arbitrary attributes, and capability-based protection [20] ties authority to unforgeable, explicitly passed tokens—the model GCP’s role credentials follow. Data-minimization and purpose-limitation obligations [21] make “read only what your role requires” a legal constraint, not merely a design preference. The point-to-point baseline

has no analogous gate: leakage prevention rests entirely on sender-side discretion.

D Leakage, provenance, and governance in agentic systems

The security literature establishes that confidentiality in agentic systems is a system property, not only a model property [22], that inter-agent message channels are the dominant leakage vector—leaking far more than final outputs and largely invisible to output-only auditing [23]—and that restricting what an agent can *see*—rather than filtering what it can *say*—is the more robust intervention [24]. The OWASP Top 10 for LLM applications [25] names Sensitive Information Disclosure and Excessive Agency as primary risks, with “embrace least privilege” as the normative mitigation. On the accountability side, deterministic privilege control at the tool boundary [26], authenticated and auditable delegation of authority to agents [27], and audit trails for LLMs [28] argue for tamper-evident, queryable records of what a system did. GCP’s contribution to this thread is to make the *per-read decision* itself the provenance primitive: every authorized or denied context read is recorded, which message-passing does not do.

Delta. Prior work supplies the pieces—read-first access (MCP), role/attribute gating (RBAC/ABAC, gated RAG), and audit trails—but not their conjunction, and not a fairness-controlled head-to-head against a real message-passing baseline. GCP combines read-first access, role gating, and audited per-read provenance, and we measure the combination against A2A.

III The Graph Context Protocol

GCP models a multi-agent ecosystem as independently owned *knowledge nodes* and *agent nodes*. Coordination is read-first: an agent acquires context by issuing a *context query* to a peer node, which returns context the caller is authorized to read. Three properties distinguish the substrate.

Read-first. Producers do not push to consumers. A knowledge node exposes context; consumers pull on demand. No agent needs to know the identity of its producers’ *consumers*, and adding a consumer requires no change at the producer. Structurally, each agent connects to the nodes it reads, not to every other agent.

Role-gated. Each knowledge node carries an access policy (`readableByRoles`, `requiredCapabilities`, `fallbackAllowed`, `denialMode`) and the server authorizes every query against the caller’s principal, defaulting to deny when no policy grants access. Authorization happens

before any content reaches the agent’s context window—the read-first analogue of complete mediation [11].

Audited. Every read decision emits a structured provenance record (`principalId`, `targetNodeId`, `timestamp`, `decision`, `matchedPolicy`) to a pluggable audit sink. This yields a complete who-read-what-when ledger for the federation, including denials.

The message-passing baseline (A2A) implements the same scenario tasks by having agents hold peer references via cards and exchange context through direct messages, with coarse card-declared allow/deny and no per-read audit sink—a faithful, good-faith control built with the same care as the GCP arm.

IV Threat and Cost Model, and Metric Definitions

We report metrics in three families.

Structural coupling (deterministic). For n peer nodes, the *pairwise connections* are the number of integration edges the topology requires. The point-to-point arm needs $C_{A2A}(n) = n(n-1)$ directed edges (every agent $\leftrightarrow$ peer pair); the federated arm needs $C_{GCP}(n) = n$ (each agent $\rightarrow$ store). *Integration effort* is the marginal cost of adding one node, $C(n+1) - C(n)$: constant 1 for GCP and $2n$ for A2A. These are properties of the topology, not of any run, so they are computed exactly.

Leakage (a lower bound). Each confidential node embeds a unique *canary* string. After a run we scan the final answer *and* the full tool transcript for exact canary matches; *leakage rate* is the fraction of a scenario’s canaries that appear where they should not. Exact-match scanning is a lower bound: paraphrased disclosure is not counted.

Provenance completeness. The fraction of read decisions for which a structured provenance record exists, clamped to $[0, 1]$. GCP records every decision; the message-passing arm has no audit sink, so its completeness is 0 by construction.

Behavioral cost and success. We count messages, connections opened, round-trips, and total tokens (summed from provider usage metadata), measure wall-clock latency, and evaluate task success with a per-scenario oracle. Counts—not just wall-clock—are the primary cost signal, since wall-clock on a shared free-tier endpoint is noisy.

V Methodology

The fairness invariant. The central methodological commitment is that the two arms differ *only* in the interop layer. A single scenario runner builds both arms from one `ScenarioDef` and one injected chat model. Both arms run the identical agent brain (a ReAct agent with one per-peer tool), the identical system prompt, goal, and success oracle. The GCP arm hosts knowledge nodes in-process behind the real server’s fetch handler with a static-token auth provider and an in-memory audit sink; the A2A arm hosts the same nodes as real localhost A2A servers. Only the per-peer tool factory and node hosting change. This removes the confound that makes most architecture comparisons uninterpretable.

Scenarios. *software-org* (leakage): a contractor-role agent must summarize public information about a project; a confidential engineering node carrying a canary is readable only by the engineering role. *supply-chain* (cross-owner): a logistics agent must state the shareable shipping relationship between two independently owned nodes (a supplier and a manufacturer), each holding an owner-scoped canary. *marketplace* (coupling): n seller nodes with descending prices; the agent must identify the cheapest. The marketplace is parameterized by n and drives the structural-coupling curve.

Model, seeds, harness. All behavioral runs use `openai/gpt-oss-120b:free` via OpenRouter at temperature 0.7, with five repetitions (seeds) per arm per scenario. The harness is gated behind an environment flag and an API key so it never runs in CI; it throttles between runs and retries on transient rate-limit errors, both of which are common on free-tier endpoints. We report mean $\pm$ population standard deviation. The structural-coupling curve is computed deterministically for $n = 2, \dots, 50$.

VI Results

A Coupling scales as $\mathcal{O}(n)$ vs. $\mathcal{O}(n^2)$

Table 1 shows the structural-coupling curve. GCP’s pairwise connections grow linearly (n) while A2A’s grow quadratically ($n(n-1)$): identical at $n=2$, $3\times$ at $n=3$, $9\times$ at $n=10$, and $49\times$ at $n=50$ (50 vs. 2450). The marginal cost of adding a node—integration effort—is constant 1 for GCP and $2n$ for A2A, so the gap *widens* with every node added.

B Provenance is complete for gcp and absent for a2a

Across all three scenarios and all five seeds, provenance completeness is 1.0 for GCP and 0.0 for A2A (Table 2). This is architectural, not statistical: GCP records a decision for

Table 1. Structural coupling (pairwise integration edges) by ecosystem size n . Deterministic.

n	GCP (n)	A2A ($n(n-1)$)	ratio A2A/GCP
2	2	2	1.0 $\times$
3	3	6	2.0 $\times$
5	5	20	4.0 $\times$
10	10	90	9.0 $\times$
20	20	380	19.0 $\times$
50	50	2450	49.0 $\times$

every read; the message-passing arm has no audit sink, so no access trail exists to query. For any governance regime that requires traceability [25, 28], this is the sharpest qualitative difference between the substrates.

C gcp never costs more, and costs less on the cross-owner task

Table 2 reports behavioral metrics. On *software-org* the two arms are within noise on tokens (678.4 ± 5.8 vs. 685.4 ± 3.4). On *supply-chain* GCP uses fewer tokens with far lower variance (707.6 ± 3.5 vs. 901.4 ± 229.2) and fewer messages/round-trips (1.0 ± 0.0 vs. 1.4 ± 0.5): the message-passing arm sometimes opened extra connections to assemble the cross-owner answer, while the federated arm read what it needed in one pass. Task success is 1.0 on both arms for these two scenarios—the equal-success precondition of the thesis holds.

D Leakage

The leakage picture requires care. On *supply-chain*, where the logistics role is authorized to read both owners’ nodes, the message-passing arm leaked every owner-scoped canary (rate 1.0) while GCP leaked half on average (0.5); because both arms are authorized to read the content, this gap reflects how each substrate surfaces content to the model rather than a gating decision. On *software-org*, the cleanest gating case, the *observed* leakage was 0.0 on both arms—but for an instructive reason: with this model the agent satisfied the public-summary task by querying only the public node (messages = 1.0) and never requested the confidential node, so the leaky path was not exercised. GCP would have *denied* that read by construction had it occurred (and recorded the denial); the message-passing arm’s coarse card would have answered it. The architectural leak is thus real but *latent* in this run, and we do not claim an observed behavioral leakage win on *software-org*. The structural guarantee—deny-by-default plus audited denial—is what GCP provides regardless of whether a given model happens to probe the boundary. The canary metric *measures* this bound; the formal model and executable property suite described in Section E *prove* it.

Table 2. Behavioral metrics, mean $\pm$ population s.d. over 5 seeds, **gpt-oss-120b:free**. Structural columns are for the scenario’s node count ($n=2$).

scenario	arm	tokens	messages	latency (ms)	leakage	provenance
software-org	GCP	678.4 $\pm$ 5.8	1.0 $\pm$ 0.0	6268 $\pm$ 482	0.0	1.0
	A2A	685.4 $\pm$ 3.4	1.0 $\pm$ 0.0	5986 $\pm$ 1168	0.0	0.0
supply-chain	GCP	707.6 $\pm$ 3.5	1.0 $\pm$ 0.0	6278 $\pm$ 1632	0.5	1.0
	A2A	901.4 $\pm$ 229.2	1.4 $\pm$ 0.5	7718 $\pm$ 2678	1.0	0.0
marketplace [†]	GCP	2425 $\pm$ 1001	2.8 $\pm$ 0.7	12617 $\pm$ 3757	0.0	1.0
	A2A	2431 $\pm$ 1082	2.8 $\pm$ 0.7	11941 $\pm$ 4304	0.0	0.0

[†] marketplace rows aggregate $n \in \{2, 5, 10\}$; success was 0.3 $\pm$ 0.5 on both arms (see Section VII). Use the structural curve (Table 1), not these mixed- n behavioral rows, for the coupling claim.

E Formal model and executable no-leak property

Access model. A knowledge node carries an `AccessPolicyDescriptor` P with two gating fields: *readableByRoles* (a list of role ids) and *requiredCapabilities* (a list of capability ids). A principal K holds an optional role with its own and inherited capability set *effective*(K), computed by `getEffectiveCapabilities` with own-first precedence when the same capability id appears at multiple levels of the role chain. The admission predicate is:

$$\text{authorized}(P, K) \equiv \underbrace{(P.\text{readableByRoles} = [] \vee K.\text{role.id} \in P.\text{readableByRoles})}_{\text{roleOk}} \wedge \underbrace{(P.\text{requiredCapabilities} = [] \vee P.\text{effectiveCapabilities} \supseteq P.\text{requiredCapabilities})}_{\text{capOk}}$$

An absent or schema-invalid policy descriptor is treated as a denial (default-deny): the gate is closed whenever a valid policy is not available.

No-leak theorem (informal). For all P and K : if $\neg \text{authorized}(P, K)$ then no `context-query` directed at a node carrying P returns that node’s content to K . The argument follows from the handler pipeline in `createContextQueryHandler`: authorization (step 4) precedes any knowledge-source adapter call (step 5). On failure the handler returns a `denied` result containing only the denial reason; the adapter is never invoked and the node’s content never enters the response.

Executable property suite. We implement three `fast-check` property-based tests over the *real* shipped code path at CI defaults of `FC_NUM_RUNS = 1000`; all pass.

P1 (end-to-end no-content-leak). For each randomly generated (P, K) pair, the property wires a live `createContextQueryHandler()` instance with a stub adapter that unconditionally returns a sentinel string `SECRET-CANARY-...` in its `raw` field and an allow-all auth provider (so the provider term drops out). Two directions are asserted simultaneously: *safety* — when $\neg \text{authorized}(P, K)$, the full JSON serialisation of the

handler response does not contain the sentinel and `payload.status` is `"denied"`; and *liveness* — when $\text{authorized}(P, K)$, the sentinel *does* appear, confirming the harness is capable of leaking and is not vacuously passing.

P2 (gate soundness). For each randomly generated (P, K) pair, the real `authorizeKnowledgeNodeAccess` function is called against a node carrying P with an allow-all provider, and the result is asserted to equal $\text{authorized}(P, K)$ computed by the independent reference oracle. This establishes that the reference predicate is an accurate specification of the shipped gate, the key compositional link that lets P1 use *authorized* as its oracle.

P3 (role-inheritance soundness). Random chains of roles (depth 2–4) over a fixed pool of five capability ids are generated and the completeness and own-first-precedence invariants of `getEffectiveCapabilities` are verified. P3 grounds the effective-capability computation used by both the reference predicate and the real gate.

Canary measurement vs. formal proof. The canary metric (Section V) quantifies how often leakage occurs across production-like scenarios; it is a *measured* lower bound. The property suite here is a *proof-level* complement: it proves that the no-leak bound holds over the entire randomised input space (subject to finite generator pools). The canary metric can detect a regression if the gate is bypassed; the property suite explains why no leak should occur and gives a constructive account of the invariant. Together they form a two-layer assurance: the property proves the gate is sound; the canary confirms the gate is on the hot path in production-like scenarios.

VII Discussion and Threats to Validity

Single model, five seeds. The behavioral results use one free-tier model (**gpt-oss-120b**) over five seeds. We therefore present them as a pilot. The structural-coupling and provenance-completeness results do *not* depend on the

model—the former is deterministic, the latter architectural—so they are robust to this limitation; the token and latency results should be replicated across models before being treated as general.

Latent vs. observed leakage. As Section VI details, the software-org leak in GCP’s favor was not behaviorally observed because the model did not probe the confidential node. This is a genuine limitation of behavioral leakage measurement: it depends on the agent exercising the path. We mitigate by also reporting the architectural guarantee and the provenance of denials, and the harness includes a forced-query hermetic test that exercises the leaky path directly.

Leakage is a lower bound. Exact-match canary scanning misses paraphrase and partial disclosure. Reported leakage rates therefore understate true exposure for both arms equally.

Marketplace task difficulty. Marketplace task success was low (0.3) for *both* arms: `gpt-oss-120b` often failed to identify the cheapest seller. Because the failure is equal across arms it does not threaten fairness, but it means the marketplace contributes through its deterministic structural curve, not its behavioral rows.

Free-tier noise. Wall-clock latency on a shared free endpoint is high-variance; we lead with counts (messages, tokens) for cost and treat latency as secondary.

Construct validity of the baseline. A weak baseline would make the comparison vacuous. The A2A arm runs real servers, the same brain, and the same task, and was built with equal care; its disadvantages (no read gate, no audit sink, pairwise topology) are intrinsic to message-passing, not artifacts of a strawman.

VIII Limitations and Future Work

The evaluation should be extended across multiple models and more seeds, with significance testing on the token and latency deltas. One designed-but-unevaluated capability remains: a stricter-capability *task-delegation* layer on the same read-first path. The formal model with an executable no-leak property is now complete (Section E), upgrading the leakage argument from measured to proven. Bridging GCP to MCP and A2A as first-class interop (beyond the benchmark baseline) is the natural deployment path.

IX Conclusion

We argued, and gave preliminary fairness-controlled evidence, that read-first role-gated context federation beats

point-to-point message-passing for multi-agent LLM systems on coupling, leakage, and cost at equal task success. Coupling improves from $\mathcal{O}(n^2)$ to $\mathcal{O}(n)$ with constant marginal integration effort; provenance is complete by construction rather than absent; and cost is never worse and sometimes better. The architecture turns leakage prevention from a matter of sender discretion into a deny-by-default, audited read gate. The strongest results—coupling and provenance—are model-independent; the cost and leakage results are a model-specific pilot that invites replication.

References

- [1] Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Shaokun Zhang, Erkan Zhu, Beibin Li, Li Jiang, Xiaoyun Zhang, and Chi Wang. AutoGen: Enabling next-gen LLM applications via multi-agent conversation. *arXiv preprint arXiv:2308.08155*, 2023. URL <https://arxiv.org/abs/2308.08155>.
- [2] Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, Juyuan Xu, Dahai Li, Zhiyuan Liu, and Maosong Sun. ChatDev: Communicative agents for software development. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*, 2024. URL <https://arxiv.org/abs/2307.07924>.
- [3] Sirui Hong, Mingchen Zhuge, Jonathan Chen, Xiwu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. MetaGPT: Meta programming for a multi-agent collaborative framework. In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024. URL <https://arxiv.org/abs/2308.00352>.
- [4] Google and Linux Foundation. Agent2Agent (A2A) protocol specification, v1.0. Open standard, 2025. URL <https://a2a-protocol.org/v1.0.0/specification/>.
- [5] Tim Finin, Rich Fritzson, Don McKay, and Robin McEntire. KQML as an agent communication language. In *Proceedings of the Third International Conference on Information and Knowledge Management (CIKM)*, 1994.
- [6] Foundation for Intelligent Physical Agents. FIPA ACL message structure specification. Technical Report SC00061G, FIPA, 2002. URL <http://www.fipa.org/specs/fipa00061/>.
- [7] Gregor Hohpe and Bobby Woolf. *Enterprise Integration Patterns: Designing, Building, and Deploying*

Messaging Solutions. Addison-Wesley Professional, 2003.

- [8] David L. Parnas. On the criteria to be used in decomposing systems into modules. *Communications of the ACM*, 15(12):1053–1058, 1972. doi: 10.1145/361598.361623.
- [9] W. P. Stevens, G. J. Myers, and L. L. Constantine. Structured design. *IBM Systems Journal*, 13(2):115–139, 1974. doi: 10.1147/sj.132.0115.
- [10] Lionel C. Briand, John W. Daly, and Jürgen K. Wüst. A unified framework for coupling measurement in object-oriented systems. *IEEE Transactions on Software Engineering*, 25(1):91–121, 1999. doi: 10.1109/32.748920.
- [11] Jerome H. Saltzer and Michael D. Schroeder. The protection of information in computer systems. *Proceedings of the IEEE*, 63(9):1278–1308, 1975. doi: 10.1109/PROC.1975.9939.
- [12] Ravi S. Sandhu, Edward J. Coyne, Hal L. Feinstein, and Charles E. Youman. Role-based access control models. *IEEE Computer*, 29(2):38–47, 1996. doi: 10.1109/2.485845.
- [13] Taicheng Guo, Xiuying Chen, Yaqi Wang, Ruidi Chang, Shichao Pei, Nitesh V. Chawla, Olaf Wiest, and Xiangliang Zhang. Large language model based multi-agents: A survey of progress and challenges. In *Proceedings of the Thirty-Third International Joint Conference on Artificial Intelligence (IJCAI)*, pages 8048–8057, 2024. URL <https://arxiv.org/abs/2402.01680>.
- [14] Anthropic. Introducing the model context protocol. Anthropic; open specification, 2024. URL <https://www.anthropic.com/news/model-context-protocol>.
- [15] Guibin Zhang, Yanwei Yue, Zhixun Li, Sukwon Yun, Guancheng Wan, Kun Wang, Dawei Cheng, Jeffrey Xu Yu, and Tianlong Chen. Cut the crap: An economical communication pipeline for LLM-based multi-agent systems. *arXiv preprint arXiv:2410.02506*, 2024. URL <https://arxiv.org/abs/2410.02506>.
- [16] Yuzhen Mao and Azalia Mirhoseini. Decentralized multi-agent systems with shared context. *arXiv preprint arXiv:2606.10662*, 2026. URL <https://arxiv.org/abs/2606.10662>.
- [17] Sebastiano Panichella, Mohammad Imranur Rahman, and Davide Taibi. Structural coupling for microservices. In *Proceedings of the 11th International Conference on Cloud Computing and Services Science (CLOSER)*, pages 280–287, 2021. doi: 10.5220/0010481902800287. URL <https://arxiv.org/abs/2103.04674>.
- [18] Xu Shen, Yixin Liu, Yiwei Dai, Yili Wang, Rui Miao, Yue Tan, Shirui Pan, and Xin Wang. Understanding the information propagation effects of communication topologies in LLM-based multi-agent systems. *arXiv preprint arXiv:2505.23352*, 2025. URL <https://arxiv.org/abs/2505.23352>.
- [19] Vincent C. Hu, David Ferraiolo, Rick Kuhn, Adam Schnitzer, Kenneth Sandlin, Robert Miller, and Karen Scarfone. Guide to attribute based access control (ABAC) definition and considerations. Technical Report NIST Special Publication 800-162, National Institute of Standards and Technology, 2014.
- [20] Jack B. Dennis and Earl C. Van Horn. Programming semantics for multiprogrammed computations. *Communications of the ACM*, 9(3):143–155, 1966. doi: 10.1145/365230.365252.
- [21] European Parliament and Council. Regulation (EU) 2016/679 (general data protection regulation), articles 5 and 25. Official Journal of the European Union, L 119, 2016. URL <https://gdpr-info.eu/>.
- [22] Jonathan Evertz, Merlin Chlosta, Lea Schönherr, and Thorsten Eisenhofer. Whispers in the machine: Confidentiality in agentic systems. *arXiv preprint arXiv:2402.06922*, 2024. URL <https://arxiv.org/abs/2402.06922>.
- [23] Fatima El Yagoubi, Godwin Badu-Marfo, and Ranwa Al Mallah. AgentLeak: A full-stack benchmark for privacy leakage in multi-agent LLM systems. *arXiv preprint arXiv:2602.11510*, 2026. URL <https://arxiv.org/abs/2602.11510>.
- [24] Eugene Bagdasarian, Ren Yi, Sahra Ghalebikesabi, Peter Kairouz, Marco Gruteser, Sewoong Oh, Borja Balle, and Daniel Ramage. AirGapAgent: Protecting privacy-conscious conversational agents. *arXiv preprint arXiv:2405.05175*, 2024. URL <https://arxiv.org/abs/2405.05175>.
- [25] OWASP GenAI Security Project. OWASP top 10 for large language model applications. OWASP Foundation, 2025. URL <https://owasp.org/www-project-top-10-for-large-language-model-application/s/>.
- [26] Tianneng Shi, Jingxuan He, Zhun Wang, Hongwei Li, Linyu Wu, Wenbo Guo, and Dawn Song. Progent: Programmable privilege control for LLM agents. *arXiv preprint arXiv:2504.11703*, 2025. URL <https://arxiv.org/abs/2504.11703>.
- [27] Tobin South, Samuele Marro, Thomas Hardjono, Robert Mahari, Cedric Deslandes Whitney, Dazza Greenwood, Alan Chan, and Alex Pentland. Authenticated delegation and authorized AI agents. *arXiv*

preprint arXiv:2501.09674, 2025. URL <https://arxiv.org/abs/2501.09674>.

- [28] Victor Ojewale, Harini Suresh, and Suresh Venkatasubramanian. Audit trails for accountability in large language models. *arXiv preprint arXiv:2601.20727*, 2026. URL <https://arxiv.org/abs/2601.20727>.